RB-602: Kubernetes Node NotReady

Field	Value
Service	Kubernetes worker nodes, cluster prod-eu-1
Severity	SEV-3 (SEV-2 if 3 or more nodes are NotReady)
Owner	Kubernetes Platform team
Last reviewed	2026-07-29

1. Symptoms

- Alert KubeNodeNotReady fires when a node reports Ready=False or Ready=Unknown for 5 minutes.
- Alert KubeNodePressure fires when a node has DiskPressure, MemoryPressure or PIDPressure set to True for 10 minutes.
- Pods on the node show Terminating or Unknown; after 300 seconds (the default not-ready toleration) they are rescheduled elsewhere.

2. Node pools

The cluster prod-eu-1 has two node pools. general-pool runs 18 nodes of type m6i.2xlarge (8 vCPU, 32 GiB) spread over 3 availability zones. memory-pool runs 6 nodes of type r6i.4xlarge for caches and JVM workloads. Nodes are named k8s-worker-NN and each belongs to an auto scaling group called prod-eu-1-<pool>-<zone>.

3. Diagnosis

3.1 Read the node conditions

```
kubectl get nodes -o wide
kubectl describe node k8s-worker-07 | grep -A10 Conditions
```

Ready=Unknown with the message Kubelet stopped posting node status means the control plane has lost contact with the kubelet. Any of the pressure conditions set to True means the kubelet is alive but evicting pods.

3.2 Check the kubelet on the host

Connect through the bastion bastion-eu-1 and inspect the kubelet:

```
ssh k8s-worker-07
sudo systemctl status kubelet
sudo journalctl -u kubelet --since "30 min ago" --no-pager | tail -50
```

If the kubelet is stopped or crash-looping, run sudo systemctl restart kubelet once. If the host does not answer SSH within 60 seconds, go straight to section 4.3.

3.3 Pressure thresholds

Condition	Eviction threshold	First thing to check
DiskPressure	nodefs.available below 10%	df -h /var/lib/containerd
DiskPressure	imagefs.available below 15%	unused images: sudo crictl rmi --prune
MemoryPressure	memory.available below 500Mi	pods without memory limits (RB-601)
PIDPressure	pid.available below 5%	fork bombs, runaway shell loops

4. Remediation

4.1 Cordon and drain the node

Stop new pods from landing on the node, then move the existing ones away:

```
kubectl cordon k8s-worker-07
kubectl drain k8s-worker-07 --ignore-daemonsets --delete-emptydir-data --timeout=10m
```

A drain that stalls is usually blocked by a PodDisruptionBudget. Find it with kubectl get pdb -A and ask the owning team before going further.

Warning: do not use kubectl drain --disable-eviction or --force to get past a PodDisruptionBudget. It deletes pods without respecting the budget and can take a whole service down. Drain only one node per availability zone at a time.

4.2 Node auto-repair

The node-auto-repair controller (namespace kube-system) replaces a node that has been NotReady for 15 minutes. It repairs at most 2 nodes per hour per cluster so that a wider failure does not trigger mass replacement. Check what it is doing:

```
kubectl -n kube-system logs deploy/node-auto-repair --since=1h
```

4.3 Replace the cloud instance

If auto-repair has hit its limit or the node is still NotReady after a kubelet restart, terminate the instance and let the auto scaling group create a new one, keeping the desired capacity:

```
aws autoscaling terminate-instance-in-auto-scaling-group \
  --instance-id i-0abc1234def567890 --no-should-decrement-desired-capacity
```

The new node joins the cluster in about 6 minutes. Remove the old node object with kubectl delete node k8s-worker-07 once the instance is gone.

5. Verification

- kubectl get nodes shows the new node Ready and no node with SchedulingDisabled left over.
- kubectl get pods -A --field-selector status.phase=Pending returns nothing older than 5 minutes.

- KubeNodeNotReady has resolved in Alertmanager.

6. Escalation

Level	Who	Contact
L1	On-call SRE	#ops-oncall
L2	Kubernetes Platform on-call	PagerDuty: k8s-platform
L3	Cloud provider support	when 3 or more nodes fail in one zone